Application Load Balancer

Table of Contents

1. Concepts 1
2. Design 7

DRAFT — For technical review

This chapter is a work in progress, shared for technical discussion and feedback. The architecture is derived from engineering requirements and constraints, not presented as a description of any particular cloud provider's implementation.

1. Concepts

1.1. What problem does an application load balancer solve?

Assume you have clients talking to a fleet of targets. Clients need some way of discovering targets and choosing which targets to send traffic to. This is traffic management.

In the simplest practical design, service discovery maintains the target membership and makes that membership discoverable. Each client queries the target set, chooses a single target, opens a connection, and sends traffic to that target. When another connection is needed, the client selects from the currently discovered targets again. This works, but clients must handle traffic management responsibilities. Clients must discover targets, select targets, and react to membership and health changes.

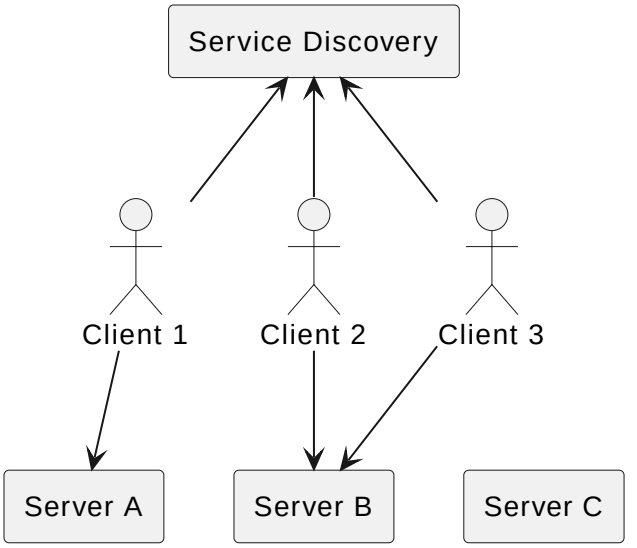


Figure 1. Clients without a load balancer

A load balancer transfers traffic-management responsibility from the clients to a reverse proxy that sits in between clients and targets. Clients discover ALB endpoints, while the load balancer maintains target membership and applies centralized traffic management policy.

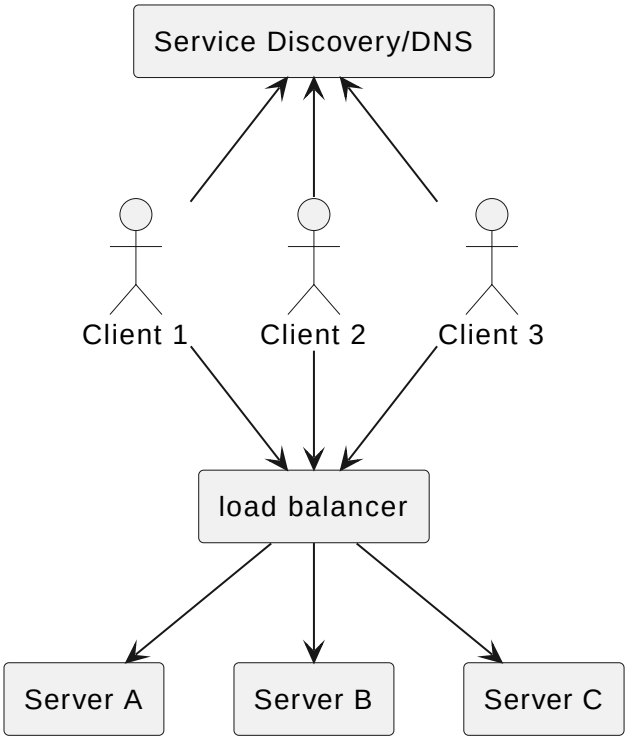


Figure 2. Clients with a load balancer

1.2. Load balancer request path

A request starts with endpoint discovery. The client resolves an ALB endpoint, establishes a connection, and, for HTTPS, completes a TLS handshake before sending the request. Once the client sends the request, the service inspects the request to determine which set of targets to route the request. A target is then selected. Unless a connection can be reused, the load balancer establishes a connection to the target. It then proxies the request. When the target responds, the response is proxied back to the client.

```
resolve service → connect to ALB → terminate TLS → receive request → inspect request
and choose target set → choose eligible target → connect to target → proxy to target
→ return response
```

1.3. DNS and endpoint discovery

The domain name system is a naming system. Among other things, it allows clients to resolve domain names to one or more IP addresses. The domain acts as a stable service name, and the IP addresses it resolves to can change.

There are two record types to be aware of: CNAME and A records. A CNAME record maps from one domain to another domain. For example, `foo.com` may have a CNAME entry to `bar.com`. DNS resolution of `foo.com` resolves to whatever `bar.com` resolves to. A records map from one domain to a set of IPv4 addresses. For example, `bar.com` may resolve to `203.0.113.10` and `203.0.113.11`.

DNS involves several layers of caching that affect consistency. When clients resolve a DNS entry, they typically communicate with resolvers that query the authoritative DNS server. Caching can occur at both the client and the resolver. Each DNS record may have a TTL that determines how long it can be cached. Caching means clients can take time to synchronize with the latest DNS changes when entries are added or removed.

1.4. TLS termination

Our application load balancer sits in front of our targets. When a request comes in, we want to route traffic according to HTTP application-layer data. If the request is encrypted (HTTPS), we need to decrypt it. Otherwise, we can't

perform application-layer routing. This is called TLS termination. If the backend requires it, we may establish another TLS connection between the load balancer and target.

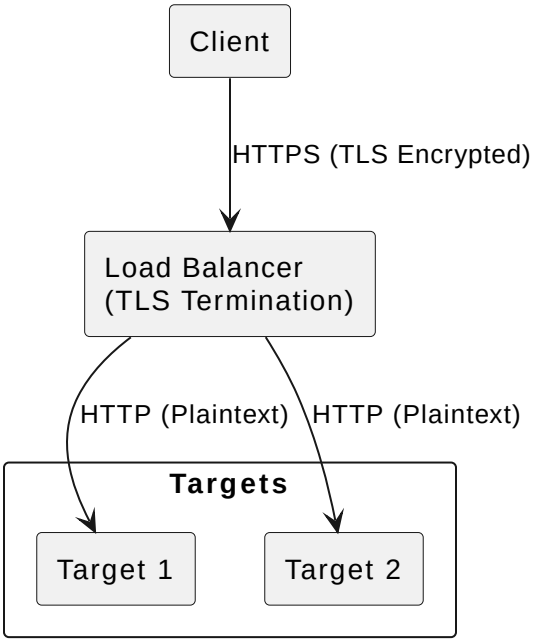


Figure 3. TLS termination example with unencrypted backend traffic

1.5. Application layer routing

When a load balancer receives a request, it must inspect it to determine which set of targets to forward the request to. An application load balancer will use HTTP metadata to make this routing decision. An HTTP request has a version, method, host, path, headers, and other metadata. Any of these could potentially determine which target set should receive the request.

```
GET /search?q=load+balancer HTTP/1.1
Host: www.domain.com
User-Agent: curl/8.0
Accept: text/html
Connection: close
```

For example, a request with the path `/delete` might be routed to a deletion service. A request with the path `/search` might be routed to a search service. An application load balancer can therefore have many target sets.

1.6. Target selection and load distribution

After a load balancer has selected a target set, it must choose an eligible target. The rule for selecting a target on each request constitutes the load distribution algorithm. The simplest algorithm might be round-robin, which we can use as the default. In educational materials about load balancers, the load distribution is often the central focus. However, for a managed cloud service, the load distribution algorithm is just a setting and not central to the architecture.

1.7. Connections and connection reuse

Client and target connections are independent entities and need not have the same lifetimes. If a client connection is terminated, the target connection is not necessarily closed, and vice versa.

Connections take time to create, increasing request latency, and so are reused across requests. However, idle connections still consume capacity. This means that requests and connections are separate workload dimensions.

ALB-target connections can be reused to send traffic from different clients. Requests from different clients may ultimately be proxied to the same targets. Reusing backend connections across clients can reduce the overall backend connection burden.

1.8. Health checks and health-aware routing

We don’t want to route traffic to unhealthy targets. Using target response codes for health checks is unreliable: the service may not know what response is appropriate for a given request, and low traffic can delay failure detection. Instead, health checks use periodic, configurable probes. To avoid flapping, we may want to wait several successive probes before changing health state. When a target becomes unhealthy, we want to stop routing traffic to it.

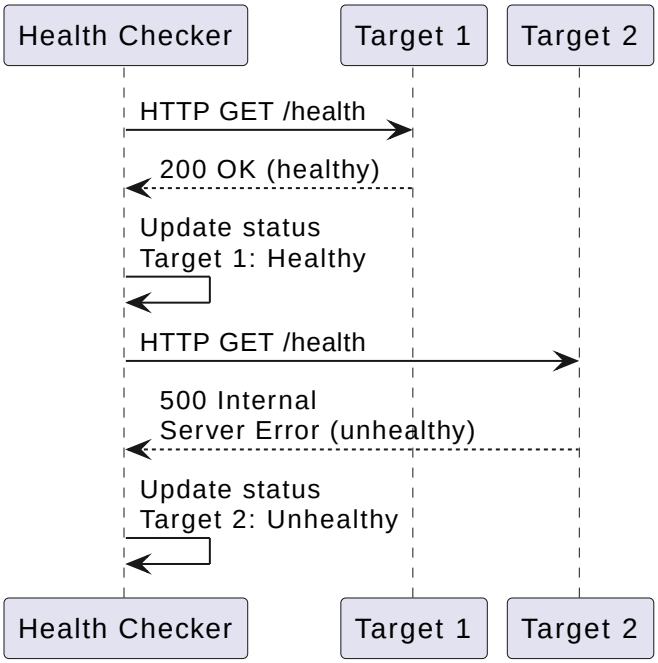


Figure 4. Target health checks

2. Design

Prompt

```
Design a regional Application Load Balancer service that routes HTTP/S requests to
registered, public IP targets.
```

2.1. Requirements and determinants

Functional requirements

- Clients send HTTP/S requests that the service distributes across registered targets.
- Tenants define rules that route requests by HTTP method, path, and headers to targets.
- Tenants register and deregister public IP targets.
- Tenants configure health checks. The service routes only to targets judged healthy if at least one target is healthy. If all targets are unhealthy, fail open by default.

Determinants

- Highly skewed utilization
- Multidimensional capacity and costs: bandwidth, connections, and request rate
- High churn target membership and health state
- Connections are long-lived and heavily reused

2.2. Derivation

Choosing the ALB execution unit

Our execution unit is a runtime unit that performs the request-processing work. For an ALB, those responsibilities are accepting a request, applying a routing decision, and proxying the request. Because bandwidth and connections are capacity bottlenecks, separating these responsibilities in

separate layers would create costly internal traffic. We therefore favor colocating them on a single node. An HTTP web server accepts HTTP/S requests, a traffic management library makes routing decisions, and an HTTP web client proxies traffic to targets.

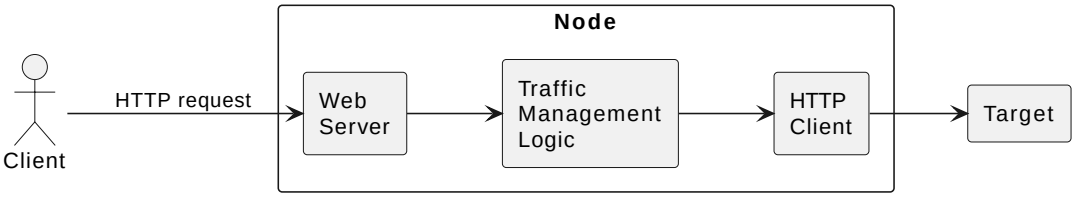


Figure 5. Application load balancer execution unit

Resource model

A load balancer resource requires an endpoint, routing rules, and targets. Giving each rule its own endpoint would duplicate endpoint infrastructure, so we allow an endpoint to have many rules. Targets routed together also share policy, such as health-check policy.

Each rule routes traffic to a single set of targets with the same shared policy. Different rules can route traffic to the same targets with the same policy. Representing those targets separately for each rule would duplicate membership and policy, and multiply the update costs of target churn. Instead we allow targets with a common policy to be reused as a destination by multiple rules. We can model this relationship by having rules reference a target set. A target set has a target policy and contains child target resources.

Rules define which traffic is selected. Target sets determine where that traffic goes and define shared policy. These concepts give us our resource model:

- Application load balancer: an endpoint
- Rule: matching criteria for an HTTP request
- Target set: logical target grouping and policy boundary
- IP target: external endpoint

A rule is operational when it's associated with a target set and the target set has at least one target.

Data plane design

Skewed utilization means many tenants don't need the full capacity of a node, while some require the capacity of many nodes. Low-utilization ALB resources favor a multi-tenant architecture in which many tenant ALBs share execution units. Potentially high-utilization ALB resources favor horizontally scalable execution units. In the simplest multi-tenant design, an ALB resource could be supported by any node in the fleet.

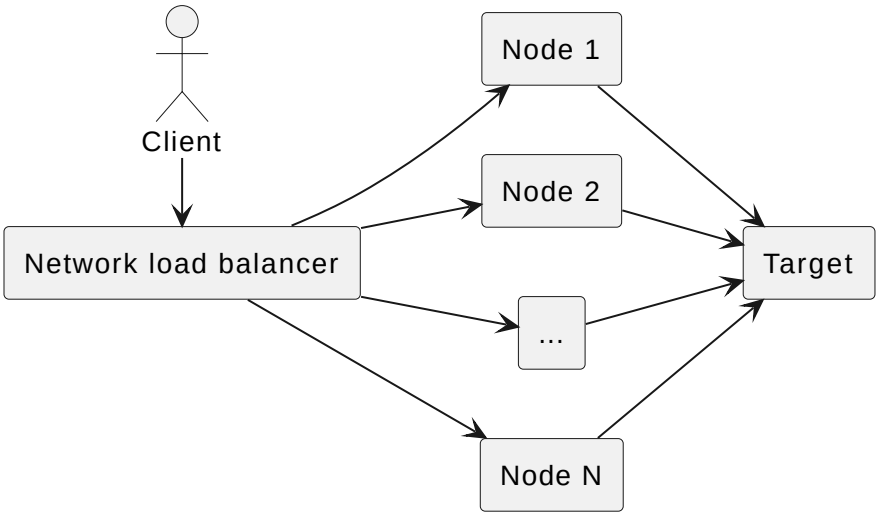


Figure 6. Simple ALB data plane design

This design has a high TCP connection burden. To understand why, we can count the total number of TCP connections needed per ALB when connections are pooled. Assume there are `C` clients, `N` nodes, and `T` targets. Each client connects to a single node, and that node connects to `T` targets. When a node connects to a target, assume only one connection is needed. When `C << N` so that separate clients do not connect to the same nodes, the total number of connections is the sum of ingress and egress connections, or `C + C * T`. When `C >> N`, each node will maintain connections to the same targets. The approximate number of egress connections each node needs is `T` with a total of `C + N * T` connections. This brings the total number of connections with pooling to `C + min(C, N) * T`. To reduce the backend connection count, we want to keep `N` as small as possible. This favors supporting each ALB with a small number of nodes.

Coupling ALB resources with supporting nodes introduces a new problem.

Node membership changes can cause massive fan-out routing updates where each ALB resource's routing must be updated individually. We can address this with shards. Each ALB is assigned to a shard, and a shard contains a small number of nodes. Shards could overlap, but sharing nodes across shards couples their failure and capacity domains, making both harder to reason about independently.

The shard acts as a stable operational scope: the shard identity persists even when its node membership changes. The use of shards allows membership, failure behavior, capacity assessment, routing, configuration distribution, and scaling to all operate against the same scope.

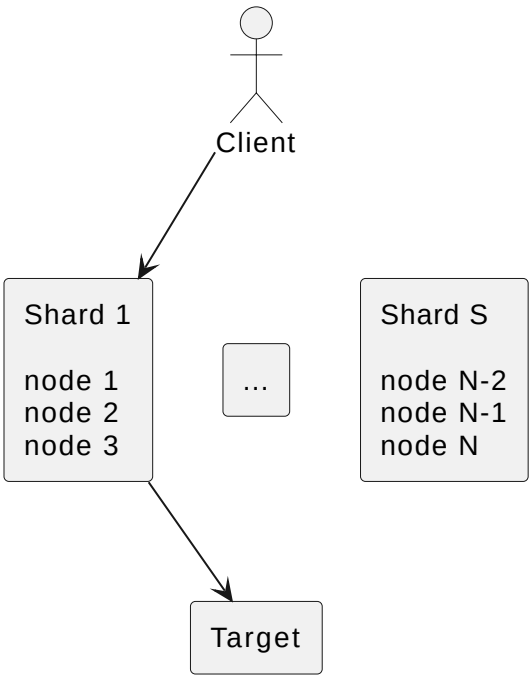


Figure 7. ALB data plane with non-overlapping shards

Data plane routing

Now that we've established that an ALB resource is placed on a shard, clients need a way to discover shard endpoints, and these endpoints must route to the shard nodes.

For clients to discover HTTP service endpoints, DNS is the natural mechanism. DNS provides human-readable endpoints, is compatible with both HTTP and

TLS, and is widely supported by clients. Each ALB can get a DNS name like `[alb_id].alb.[region].cloud.com`. The use of a single wildcard certificate, such as `*.alb.[region].cloud.com`, allows us to use TLS for any load balancer resource.

Rather than have a DNS A record which resolves directly to IP addresses, we will have the resource DNS record be a CNAME which resolves to a shard record like `[shard_id].alb.[region].cloud.com`. This shard record will be an A record and resolve directly to the shard endpoints. This approach allows us to only update a single DNS entry when shard endpoints change rather than every ALB DNS entry in the shard.

Our DNS-exposed shard endpoints could be stable VIPs or node-bound IPs. The biggest difference comes down to how the backend is selected. With stable VIPs, backend selection is done at connection time, allowing routing to react quickly to shard member changes. With node-bound IPs, each address identifies a specific node, so membership changes require DNS updates. The authoritative DNS server can be updated within seconds, but resolvers and clients may cache old addresses until their TTLs expire.

VIPs therefore provide faster shard membership convergence. However, they introduce a new data plane, an operational dependency, and a failure domain on our request critical path. Even if the VIP is provided by the cloud platform, the data-plane dependency and failure domains remain.

While DNS does have slow routing convergence, several mechanisms can mitigate its effects. Clients who see a connection failure due to a bad node can retry using another IP address. We can manage DNS staleness by providing multiple shard node IPs, implementing a short TTL, and continuing to accept traffic after node unpublishing through the DNS TTL horizon.

We therefore accept the slow, client-dependent membership convergence in exchange for avoiding an additional data-plane dependency. This leads us to prefer node-bound addresses.

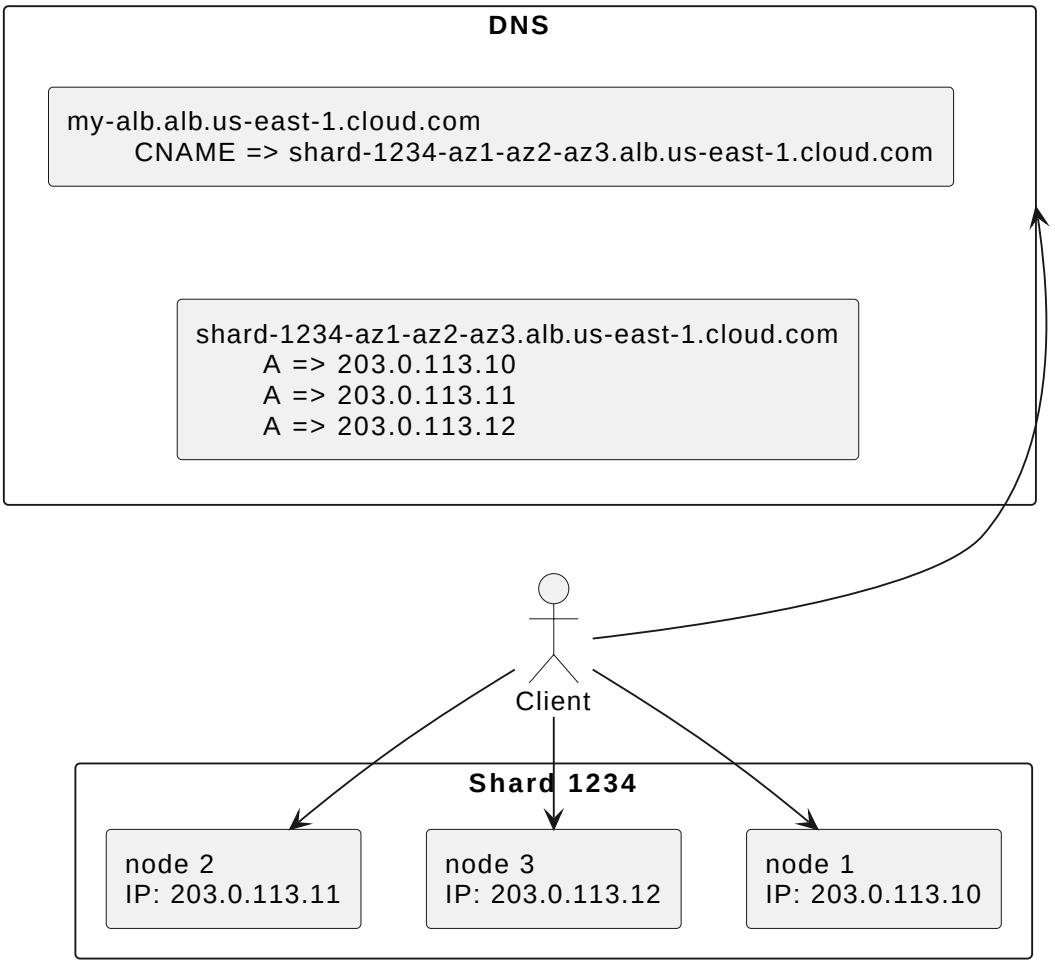


Figure 8. Simplified shard endpoint discovery and routing

The use of DNS with node-bound IPs has an important safety implication. Node membership is expected to churn, and a new node may acquire a recently released IP. If the time between release and acquisition is within the DNS TTL horizon, a client may connect to a node in a different shard. To avoid this, we should wait for the DNS TTL horizon to expire after DNS withdrawal before releasing an IP address.

Shard sizing and capacity envelope

We need to determine how many nodes are in a shard and how heavily those nodes can be utilized. Availability, practical constraints, and failure redistribution constrain the answers.

Availability creates a required topology and a lower bound on size. If an zone

fails, all the nodes in the zone will be unavailable. Therefore, shard nodes should span at least two zones to survive an zone failure. The shard nodes in each zone will be referred to as a subshard. Our canonical shard design will span all three zones. We also want to preserve a subshard's serving capacity in the face of a single node loss. This means we must have at least two nodes per subshard. This forces the minimum number of nodes in a three-zone shard to be six.

Practical system constraints limit how large a shard can become; however, they don't provide a hard upper bound. Shards become increasingly inefficient as they grow. Large shards require more node-target connection pools to support traffic. Due to DNS response size limits, only a subset of IPs may be returnable for any DNS query, and this could be a source of traffic skew. Any shard- or subshard-scoped coordination will see contention increase as shards grow. Ultimately the upper limits must be determined empirically.

Failure redistribution can force utilization and topology bounds. In a failure scenario, the remaining workload is redistributed among surviving nodes. For simplicity, we assume load is evenly distributed across nodes. The failure of a single zone will redistribute subshard traffic across the two remaining subshards. Each surviving subshard will see a 50% traffic increase. By keeping utilization of our capacity dimensions below 67%, we can redistribute the traffic without overloading the subshards. Similarly, the failure of a single node may redistribute traffic among the surviving nodes of the subshard. With two nodes per zone, utilization must stay below 50%. However, with three nodes, utilization must stay below 67%. To keep utilization high, we prefer three nodes per subshard in a three-zone shard.

This suggests a reasonable starting shard size is nine, with three nodes per zone, and a target utilization of 67%.

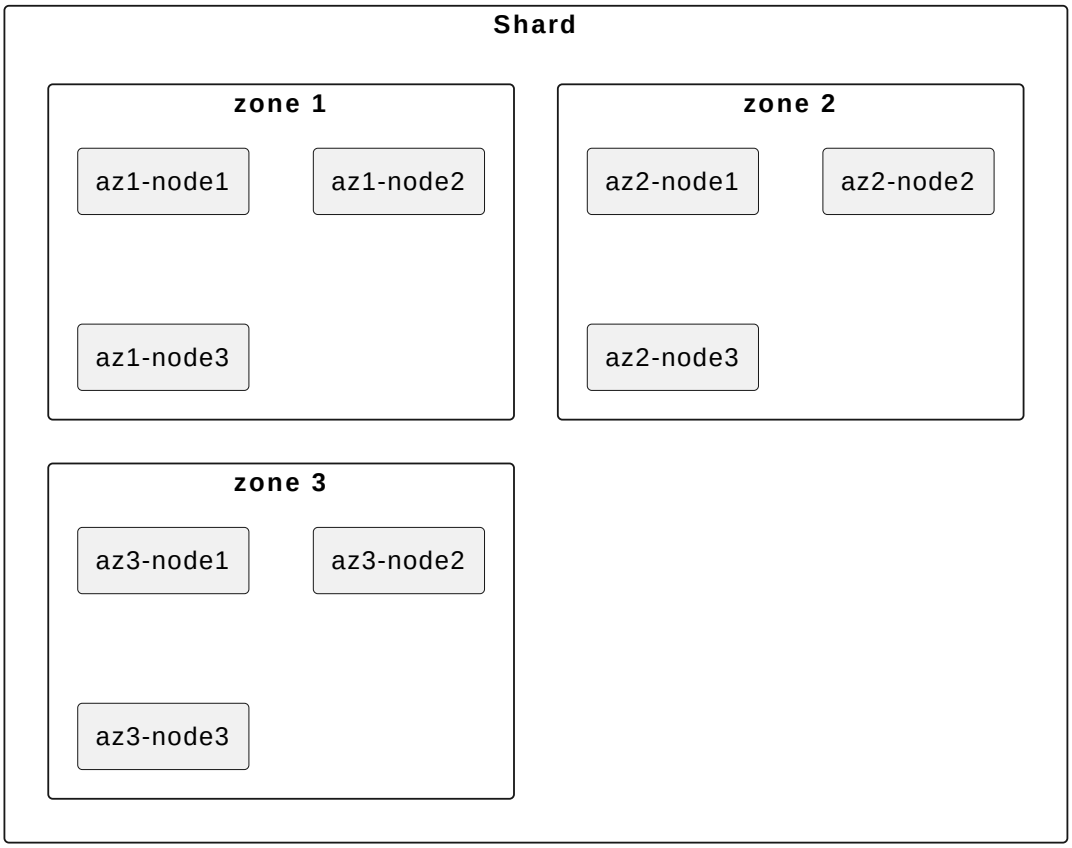


Figure 9. Example three-zone data plane shard

Configuration model

We need to provide a consistent, data-plane-ready resource representation. These are our configurations, derived from control plane resource state. Because application load balancers, rules, target sets, and targets have separate mutation patterns, separating their configurations avoids coupling unrelated updates. This gives us an ALB config, a rule config, a target set config, and target configs.

Placement of ALB resources implies a distribution scope. Because ALBs are placed on a shard, the configuration distribution scope is the shard. When bootstrapping, a node needs to pull in all the shard configurations. Rather than loading configurations incrementally, it can load the entire config set as a snapshot. The snapshot includes a generation, giving the node a starting point for consuming subsequent configuration updates.

After the node loads a snapshot, its configurations become stale as tenants update resources, so the node needs an incremental, consistent way to sync the latest updates. We can use an ordered change stream which groups related configurations into generations. Each generation will contain a consistent list of new, changed, and deleted configuration identifiers. Nodes can read the change stream and then load referenced configurations individually.

Because nodes download snapshots, change streams, and configurations independently, nodes within a shard may be synchronized to different generations at any time. However, the node advances monotonically through configuration generations.

Health-aware target routing

Target membership alone doesn't tell us if a target is reachable. Health-aware target routing allows nodes to dynamically update which targets receive traffic based on health indicators.

Using the model established earlier, a health probe is an HTTP GET request to a tenant-defined path. A `200 OK` response passes the check; any other response or failure to respond within a timeout fails the check. The tenant can configure the probe interval, the number of successful probes to mark a target as healthy, and the number of unsuccessful probes to mark a target as unhealthy.

Health checks should reflect the reachability experienced by the serving data plane. Generally speaking, health probes should originate from the same networking and failure domain as the traffic whose routing they control. This leads us to prefer implementing the probes on the nodes themselves rather than using a separate probing fleet.

While each node in a shard could perform health checks on every target, this creates high health-check overhead. We can strike a balance between probe costs and probe granularity by having one node perform health checks on a given target per subshard.

Probe authority must be distributed across the subshard. Concentrating all health check probes on a single subshard node can create utilization skew

independent of traffic. It's preferable to distribute the probe workload across all subshard nodes rather than concentrate it on a single one. To partition probe authority, we use per-partition leases on a subset of targets.

This leaves the choice of partition granularity. We can partition on the tenant, ALB, target set, or target. Partitioning on targets creates the greatest workload smoothing, but the target set is an attractive choice because it balances load granularity and state locality (shared health-check policy). To avoid hot spots when partitioning on the target set, we impose quotas to bound the number of targets.

Single-node probe authority creates another problem: other subshard nodes need to read health observations for routing. We can use shared state coordination. The subshard partition probe authorities will write health check statuses to a subshard cache. To handle stale lease holders that submit health data, we can use generation-scoped keys. The key will look like `health/{partition_id}/{lease_epoch}/{target_id}`. The reader nodes use the current partition lease epoch for fencing. If there is no observation for that epoch, the target health is `UNKNOWN`. They periodically reconcile these values with their local caches. Routing decisions will be made according to local cache values only. Using local cache values for health routing removes the subshard cache dependency from the critical path.

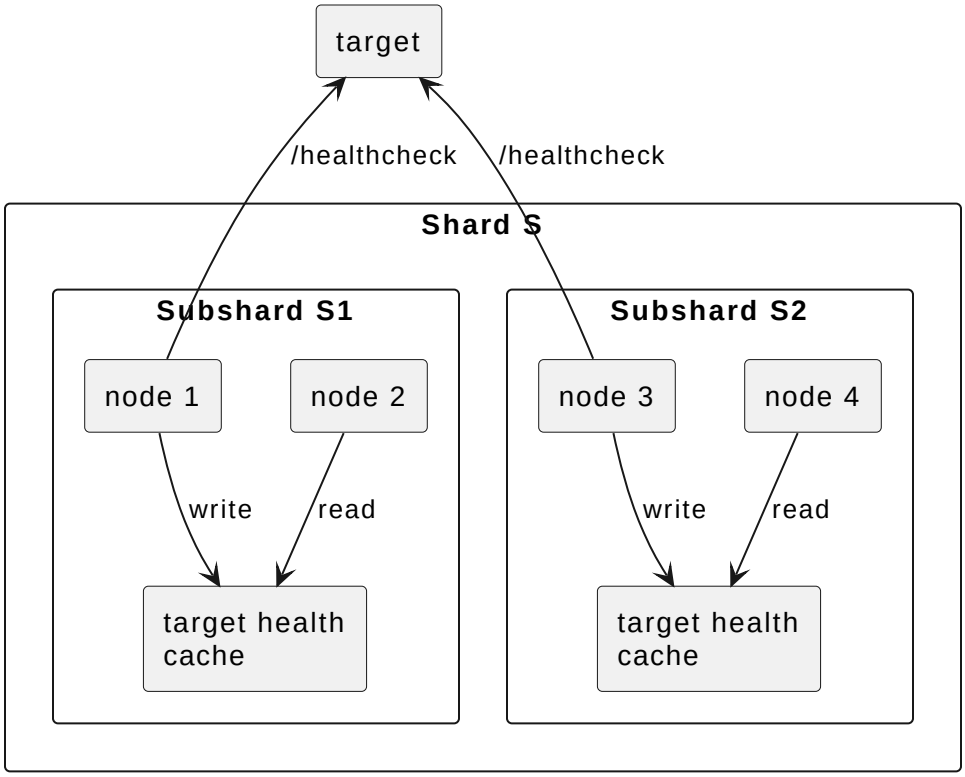


Figure 10. Shard state coordination for per-zone target routing

Failure of the target-health system creates a tradeoff between availability (continue routing) and correctness (route only to healthy targets). Recent observations can be reused briefly. After sufficient staleness, fail-open or fail-closed policy can be applied. Tenants can configure their desired failure behavior.

Data plane so far

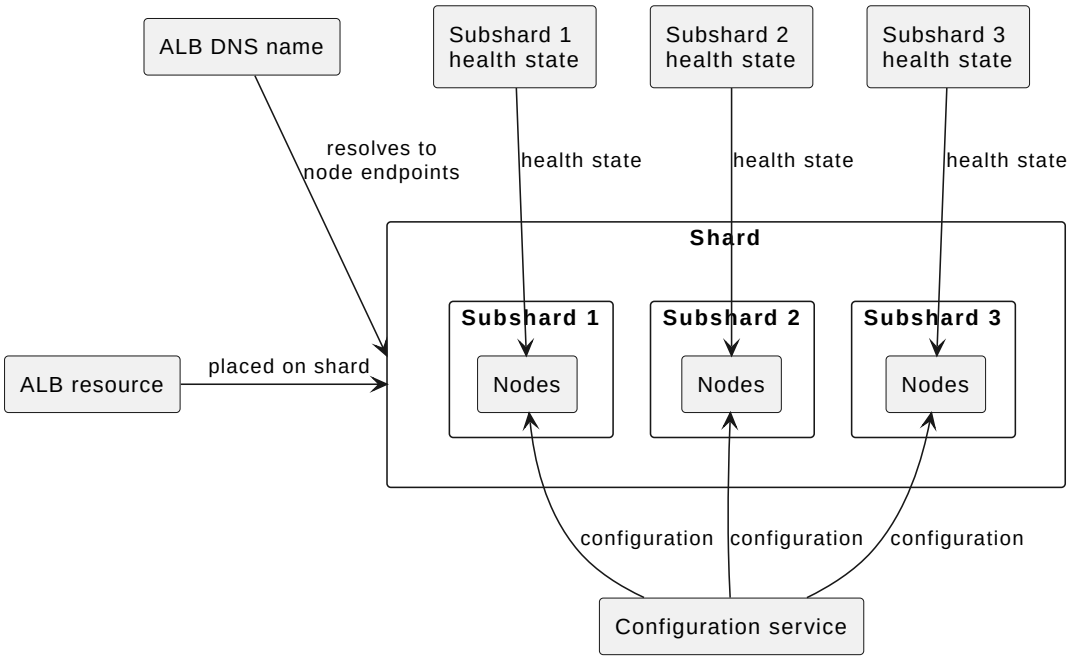
Previous sections covered shard topology, routing, and state separately. This section summarizes what we've derived about shards with a unified model.

A shard is the unit of ALB placement. It is composed of three subshards. Each subshard is composed of nodes within a single zone. The use of three subshards allows a shard to survive the loss of a single zone subject to capacity headroom in the remaining subshards.

Routing uses DNS: the ALB resource domain resolves to the node IPs in its shard. Clients select an endpoint when establishing a connection and reuse the

connection across requests. The service is responsible for updating the shard DNS endpoints based on node membership.

Nodes use two forms of state for request routing: resource configurations and target health state. Resource configurations have shard scope. Resource configurations are distributed to all nodes within the shard. Health state has subshard scope. Only nodes within the same subshard share target health state. Nodes combine resource configuration state with target health state for routing. Resource configurations are used to determine which targets are candidates, and health state is used to determine which targets are eligible, subject to fail-open or fail-closed behavior.



Target-health reporting

Tenants need visibility into the health-aware routing decisions of their ALB resources. Without this, tenants will have trouble debugging their systems when traffic doesn't flow as expected.

Probe authorities report target-health observations to the control plane. Each observation identifies the target, its health status, a timestamp, and the policy and lease versions under which the observation was produced, allowing stale observations to be rejected.

To surface health-check status, we use the standard data-plane-to-control-plane propagation path: the observation flow.

A single target in a three-zone shard has one independently computed health status per zone. The zones may not all report the same health status, so we want to surface an aggregate health status for the tenants. We can use the following aggregate health statuses: `HEALTHY`, `PARTIALLY_HEALTHY`, `UNHEALTHY`, and `UNKNOWN`.

Aggregate status	Description
`HEALTHY`	All zones report the target as healthy.
`PARTIALLY_HEALTHY`	All zones are reporting; at least one reports healthy, and at least one reports unhealthy.
`UNHEALTHY`	All zones report the target as unhealthy.
`UNKNOWN`	At least one zone is not reporting.

Our observations are high-write, ephemeral, reconstructible, and infrequently read. These workload characteristics favor optimizing the write path over the read path. Because each observation is scoped to an zone, we can write it to an zone-local cache. This allows us to avoid expensive cross-zone coordination and fan-out on the write path. The cache should be optimized for write throughput and not durability. In the event of total data loss, the cache can be reconstructed within minutes.

When tenants want a regional target health view, we do a cross-zone, fan-out read. While reads are expensive operations, they are infrequent, so we pay cross-zone costs on reads rather than writes.

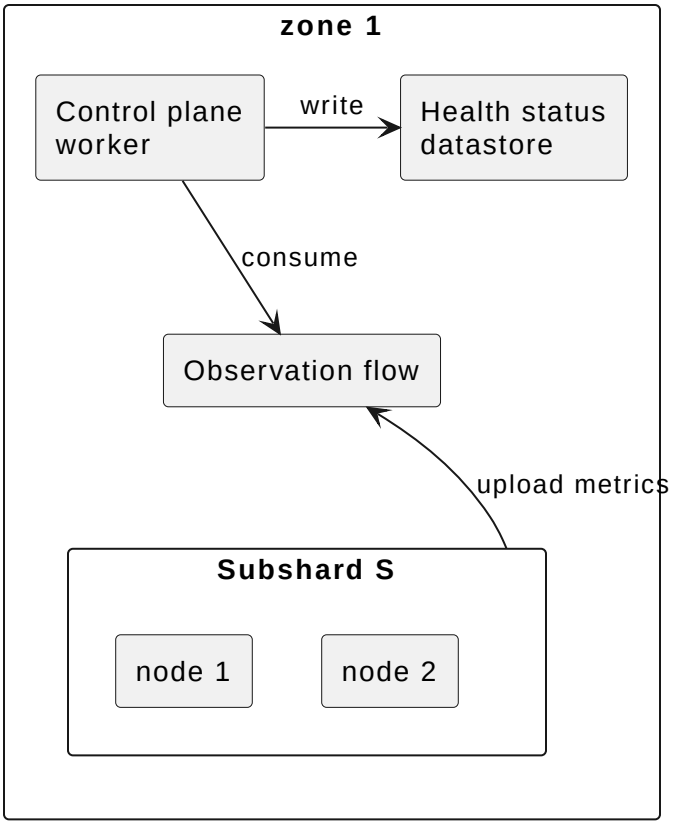


Figure 11. Health observations are stored in an zone-local cache

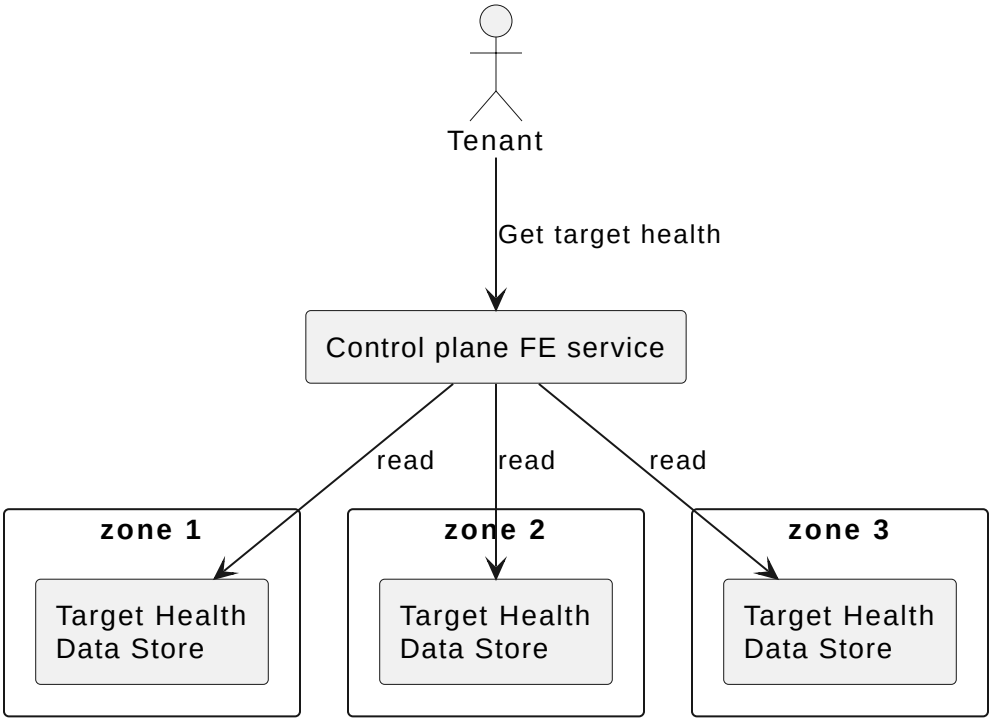


Figure 12. Target health reconstruction is a fan-out read

ALB creation, placement, and deletion

The ALB is a foundational resource. Without it, rules, target sets, and targets cannot exist. Unlike creation and deletion of the above resources, ALB creation and deletion coordinate external side effects beyond configuration distribution.

Creation and deletion share a common lifecycle model. Both require coordinating external side effects, such as asynchronous routing updates. We expose them as asynchronous operations with intermediate states. Because both operations involve a known sequence of steps for which we want a single owner, they are modeled as workflows with durable triggers. Lifecycle state is used to determine what transitions are legal. We want states to cover intermediate and completed states for creation and deletion. A starting set of states can be `CREATING`, `ACTIVE`, `DELETING`, and `DELETED`. The `CREATING` and `DELETING` states can act as durable workflow triggers.

Creation selects shard placement, creates an ALB DNS entry, and marks the resource as `ACTIVE`.

ALB placement uses max normalized utilization metric. Each shard capacity metric is converted into a numeric value by dividing current utilization by maximum allowed utilization. The maximum of these values taken and compared to other shard's max normalized utilization. For more information, see in [cells-placement-routing-placement]. The utilization dimensions are configuration snapshot size, configuration count, and shard size. Configuration snapshot size captures the per-node state which must be loaded during bootstrapping; Configuration count captures the cost of propagating configuration updates; and the shard size captures the number of target connections each ALB must maintain.

Delayed feedback affects placement mechanics. When multiple resources are placed together concurrently, their effects may not be immediately reflected in placement inputs. If all resources are placed on the globally least-loaded shard, we risk thundering herds and thrashing as placement inputs update. To avoid this, we sample a subset of shards and pick the locally least loaded. To avoid overcommitting shards, we rate limit the creation workflows and allow bounded overshoot. The alternative to bounded overshoot is capacity reservations, which can provide strictly enforced caps but at the cost of additional complexity.

Creation has a potential concurrency conflict. If two creation workflows are operating concurrently, they may race, leaving the recorded placement shard inconsistent with the ALB DNS entry. To avoid this, we need a durable boundary before side effects are processed. Placement becomes authoritative exactly once, before updating DNS. Subsequent or redundant workflow executions must use that recorded placement.

Deletion prevents child resource mutations, withdraws the ALB DNS entry, cleans up child resources, and marks the resource as `DELETED`.

To prevent child resources from being created during deletion, we use `DELETING` as a mutation gate. Child resource create and update operations will commit only when the ALB resource state is `ACTIVE`. Once in `DELETING`, the child resource set will only shrink, allowing the workflow to remove all remaining children before marking the resource as `DELETED`.

Node lifecycle and DNS publishing

The nodes supporting a shard are constantly churning. OS updates, security patches, and failures can cause nodes to be replaced. As the nodes change, the shard DNS entry must be updated.

Invariants constrain the structure of the node lifecycle. We consider steady-state conditions, node membership addition/removal, and failure modes.

- The published shard DNS entry converges to the set of ready, non-retiring nodes.
- A new node should eventually be published in the shard DNS entry.
- A new node must download configurations, acquire an IP address, and stand up the execution environment before its IP is published.
- A retiring node should eventually be unpublished from the shard DNS entry.
- A retiring node should be unpublished before draining connections, releasing the IP address, and terminating the node.
- An unhealthy node is eventually retired.
- A failed node is eventually unpublished and the IP released.
- An IP should not be eligible for reassignment until all stale DNS references to the previous shard have expired.

Determining when a node is ready only requires node-local state: downloading configurations, acquiring an IP address, and standing up the execution environment. Shard DNS publication requires a different view: the readiness and IP addresses of nodes across the shard. Node and DNS resources operate at different scopes and should have separate authorities. A consequence of a group-level authority is the need for an authoritative membership set. A DNS controller needs an authoritative view of which nodes are eligible for publication. This is our shard membership set. Shard membership connects node-level eligibility and shard-level publication. Because node creation/retirement has ordering requirements with DNS updates, the node lifecycle must coordinate with DNS publication progress.

Shard membership and DNS publishing are separate concerns. Membership is authority and DNS is derived state. Nodes declare membership eligibility, and a DNS controller reconciles that state. Nodes become eligible to serve traffic once they are ready and become ineligible when retirement is requested. Because DNS updates are asynchronous, the desired and published RRSet may differ, and so a DNS controller continuously reconciles the published state toward the desired state.

Node activation and retirement use opposite publication ordering. A new node acquires an IP address, downloads configurations, and stands up the execution environment prior to becoming eligible for authoritative shard membership. The DNS controller then publishes it. Retirement reverses the sequence: the node requests retirement, making itself ineligible for membership, waits for DNS withdrawal and for the DNS TTL horizon to elapse, then drains connections and releases its IP.

An unhealthy but responsive node shares the same lifecycle as graceful retirement. A monitoring system detects the node is unhealthy, and then triggers the retirement path.

Abrupt node loss uses the same membership and DNS withdrawal mechanisms as graceful retirement. The main difference is that the authority must be transferred since the node is not present to orchestrate actions. Monitoring is needed to detect abrupt node loss, and a separate authority is needed to initiate membership withdrawal, wait for the DNS TTL horizon to expire, and release the IP address.

The fleet management service and node lifecycle scripts implement the normal node creation and retirement paths. The fleet management service can trigger node creation and termination based on a desired membership count and node health. Fleet management services typically allow users to execute custom scripts at designated times, such as right after node creation and right before termination. These will be our startup and termination scripts. These scripts allow us to make node-local changes like downloading configurations, setting up the execution environment, draining connections, and acquiring/releasing IPs. We avoid DNS changes in these scripts because DNS is a group-level resource and best served by a group-level controller.

Shard membership can be determined through a per-node, durable `NodeResource` entry. This entry can have flags to signal membership intent. For example, a `ready` flag can be set when a node is ready to be added, and another flag `retirement_requested` can signal retirement intent. Membership may be evaluated as a predicate such as `ready && !retirement_requested`. The `NodeResource` entry can also contain metadata used by the DNS controller, like an IP address.

DNS publication can be managed through a shard-scoped controller called the shard DNS publication reconciler. It uses desired state reconciliation to converge node membership against the shard DNS entry. It will own a per-shard `ShardDnsPublication` table entry to record desired and observed state. The DNS desired state is not authoritative but a derived view of membership. DNS operations are asynchronous, and so we depend on the DNS service to provide versioned updates or fencing tied to publication authority for safety. If the DNS service cannot provide this, a stale lease holder can potentially publish stale state, and we can only guarantee eventual convergence to the correct state.

Fleet management application and platform health checks can detect unhealthy nodes and initiate the normal retirement path if the node remains responsive. Fleet lifecycle events can be used to detect abrupt termination and transfer responsibility to a separate authority when the node cannot advance its own lifecycle. We will call this secondary authority the node termination reconciler. The node termination reconciler will respond to node termination events. For correctness, it can periodically poll events per fleet. For latency, it should subscribe to a pub/sub service. To retire a dead node, the reconciler can set the `retirement_requested` flag on `NodeResource`. Once the node is unpublished from DNS and the DNS TTL horizon has elapsed, the reconciler can clean up dangling resources such as the IP address.

IP address management is undifferentiated work, and so we depend on a public IPAM service rather than build it ourselves. It needs to provide capabilities to allocate an IP address, hold it independent of the node lifecycle, and release the IP address. Even though we don't implement it, we do need to expose a contract.

- `acquirePublicIP()`: creates a public IP and an allocation identifier

- `releasePublicIP(public_ip_allocation_id)`: idempotent release

Table 1. Shard operation responsibilities

Responsibility	Description	Implementation
Node creation and termination	Nodes must be created and destroyed	Fleet management service
Node configuration and teardown	Nodes require configurations to service traffic and must drain on shutdown. IPs must be allocated and deallocated.	Custom scripts executed on node lifecycle events
Authoritative shard membership	Create a consistent, authoritative membership set	`NodeResource` entries
DNS publication	DNS serving state should reflect authoritative members	Shard DNS publication reconciler and `ShardDnsPublication` entries
Failure detection and recovery	Detect unhealthy or missing nodes and initiate recovery actions	Fleet management health checks and event logs + node termination reconciler
IP address management	Acquire and release public IPs from a pool	Public IPAM service

Scaling and capacity

Tenant workloads are dynamic and can grow beyond the capacity of the existing nodes. We need a mechanism to adjust the capacity available to an ALB resource.

We can respond to insufficient capacity with horizontal shard scaling or shard migration. Horizontal scaling is preferable because it allows us to keep resource placement. We can also use our existing fleet management service to provide this. For extremely large tenants that are too large for a normal shard, dedicated shards may be needed, but this is out of scope.

Our shard scaling needs a scope. The obvious choices are to scale at the shard or subshard level. As a general rule, the scaling scope should coincide with the traffic allocation scope because they are coupled. Recall that each tenant ALB

DNS entry is a CNAME to a shard DNS entry. The shard DNS entry contains all the nodes in a shard. This means our traffic allocation scope is the shard, so our scaling scope should also be the shard. Shard scaling should also preserve a roughly even distribution of nodes across subshards.

When we increase the number of nodes in a shard, we see an obvious problem: new connections begin to spread across the expanded node set, but existing connections and traffic are still pinned to the old nodes. This is an inherent conflict between long-lived client connections and horizontal scaling. To counter this, we can establish a maximum connection lifetime, such as ten minutes. This allows client connections to churn and redistribute across the current node set over time.

Capacity is multidimensional, so our scaling metrics should also be multidimensional. We can configure custom metrics and alarms that the fleet management service uses to trigger upscaling and downscaling events. The metrics we use will be the various resource bottlenecks that can occur under different kinds of workloads. Node capacity is constrained by request rate, new connection rate, existing connection rate, byte processing rate, and potentially CPU for TLS termination. Because saturation in any dimension can constrain capacity, we scale up when any metric exceeds our utilization target. Scaling down is safe only when every metric has sufficient headroom.

2.3. Architecture snapshot

External APIs

The load balancer exposes standard CRUD APIs:

- `CreateALB()`
- `ListALBs()`
- `GetALB(alb_id)`
- `DeleteALB(alb_id)`

Our target set CRUD APIs must expose policy:

- `CreateTargetSet(alb_id)`
- `ListTargetSets(alb_id)`
- `GetTargetSet(alb_id, target_set_id)`
- `DeleteTargetSet(alb_id, target_set_id)`
- `UpdateTargetSet(alb_id, target_set_id, target_port, health_check_path, datapoints_until_healthy, datapoints_until_unhealthy, healthcheck_period_seconds, failure_behavior)`

Target CRUD APIs must expose health:

- `AddTargets(alb_id, target_set_id, ips)`
- `ListTargets(alb_id, target_set_id)`
- `GetTarget(alb_id, target_set_id, target_id)`
- `RemoveTargets(alb_id, target_set_id, target_ids)`
- `GetTargetHealth(alb_id, target_set_id, target_id)`

Rules have the following APIs:

- `CreateRule(alb_id, method, path, header_kvp_list, priority, target_set_id)`
- `ListRules(alb_id)`
- `GetRule(alb_id, rule_id)`
- `DeleteRule(alb_id, rule_id)`
- `UpdateRule(alb_id, rule_id, method, path, header_kvp_list, priority, target_set_id)`

The data plane exposes the request interface.

- `InvokeRequest(protocol, method, domain, path, query, headers, body)`

Table 2. Architecturally significant API semantics

API	Execution model	Semantics
`CreateALB`	Synchronous acceptance followed by a workflow	Persists the ALB in `CREATING`. The workflow persists shard placement, creates the resource DNS entry, and changes the state to `ACTIVE` after publication completes.
`DeleteALB`	Synchronous acceptance followed by a workflow	Requires an `ACTIVE` ALB. The request changes the state to `DELETING`, preventing new child creation and update; the workflow removes the DNS entry, removes remaining child resources, and changes the state to `DELETED`.
`UpdateRule`	Synchronous database update followed by configuration distribution	The resource update is accepted when committed to the resource database. It becomes effective after a configuration distribution.
`AddTargets`	Synchronous database update followed by configuration distribution and routing convergence	Target registration is accepted when committed to the resource database. A target becomes eligible for routing after configuration distribution and health checks classify it as healthy.
`GetTargetHealth`	Synchronous fan-out read	Returns the latest available regional health observations. The result may lag target registration, health-policy changes, and recent probe results.

Data model

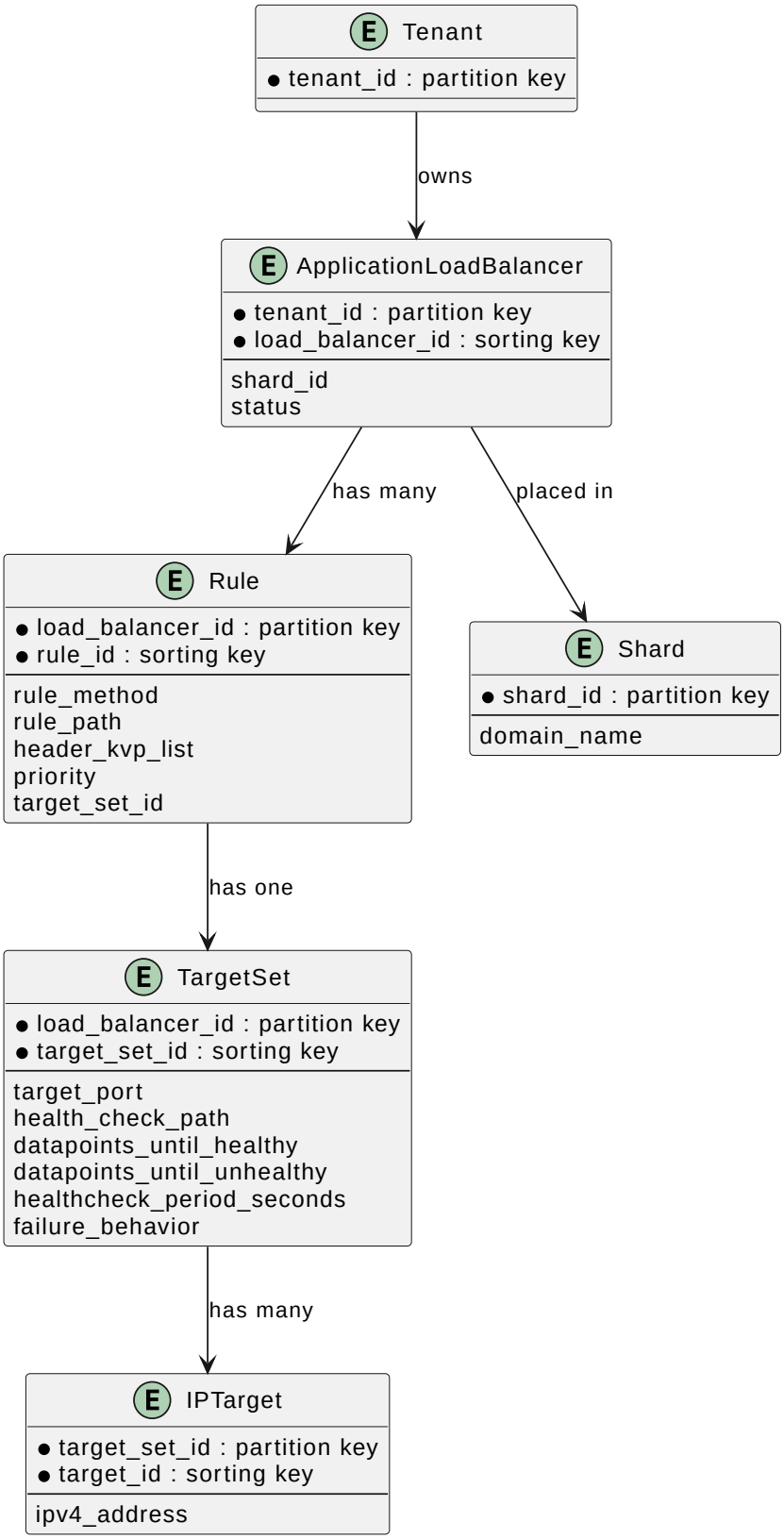


Figure 13. Resource data model

We also have data entities to support shard operations. We have separate state for nodes and the shard controller.

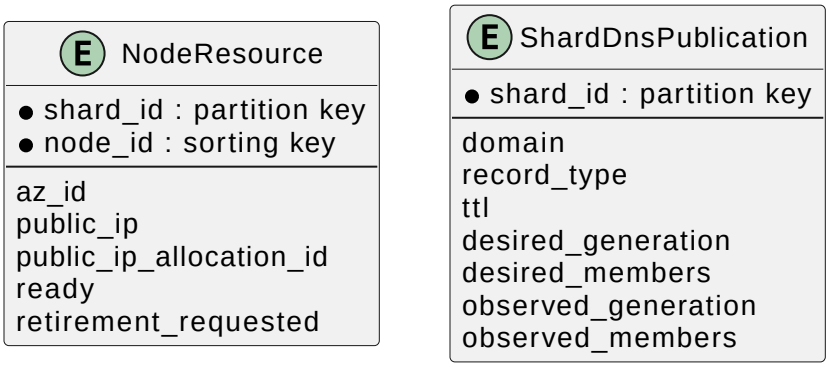


Figure 14. Shard operations data model

The target-health cache is omitted because it consists only of direct key lookups.

Table 3. State ownership

Owner/writer	State
Control plane FE service	`ApplicationLoadBalancer.status`, `Rule`, `TargetSet`, `IPTarget`
Node lifecycle scripts	`NodeResource`
Shard DNS publication reconciler	`ShardDnsPublication`
ALB lifecycle workflows	`ApplicationLoadBalancer`
Node termination reconciler	`NodeResource.retirement_requested`
Configuration service	ALB configs, rule configs, and target set configs

Architecture

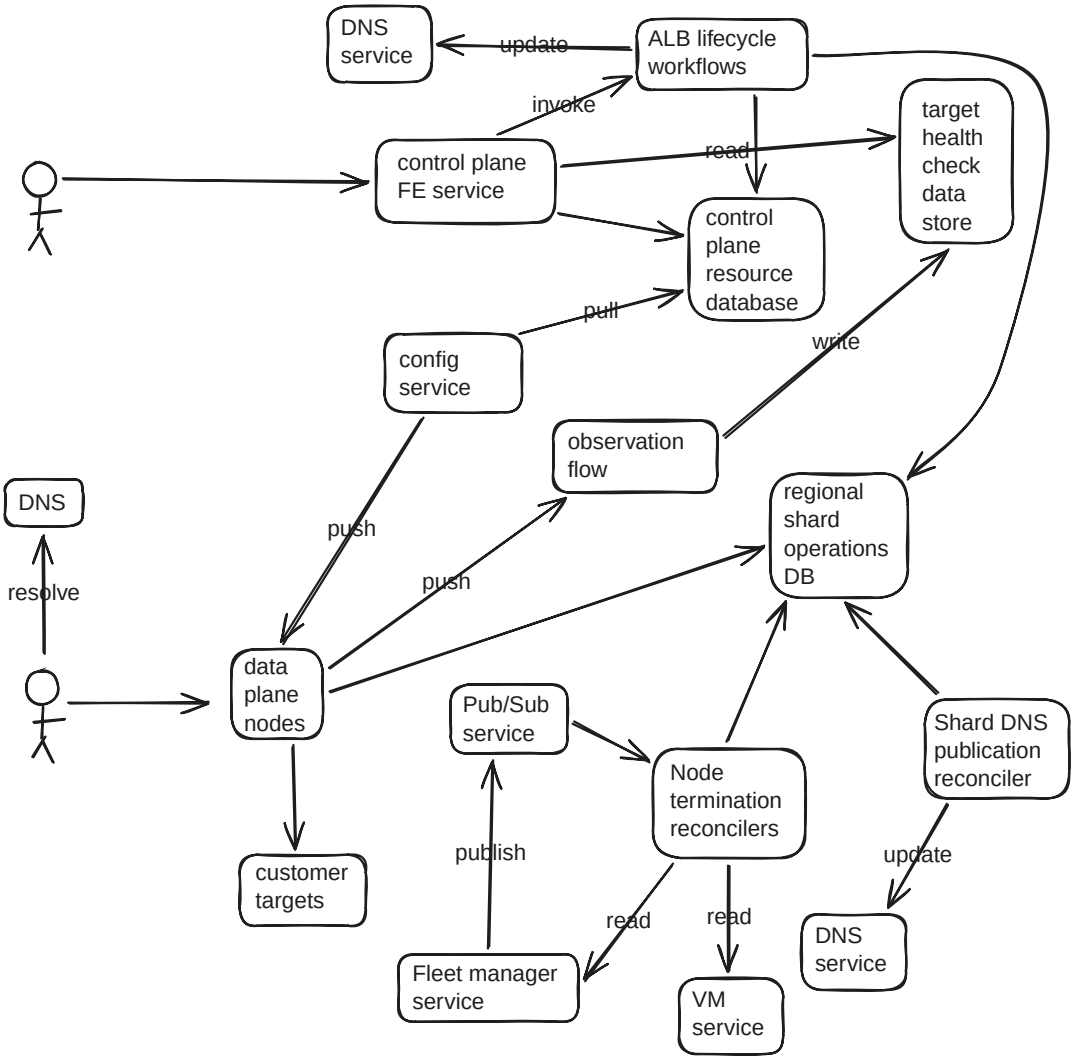


Figure 15. Load balancer architecture

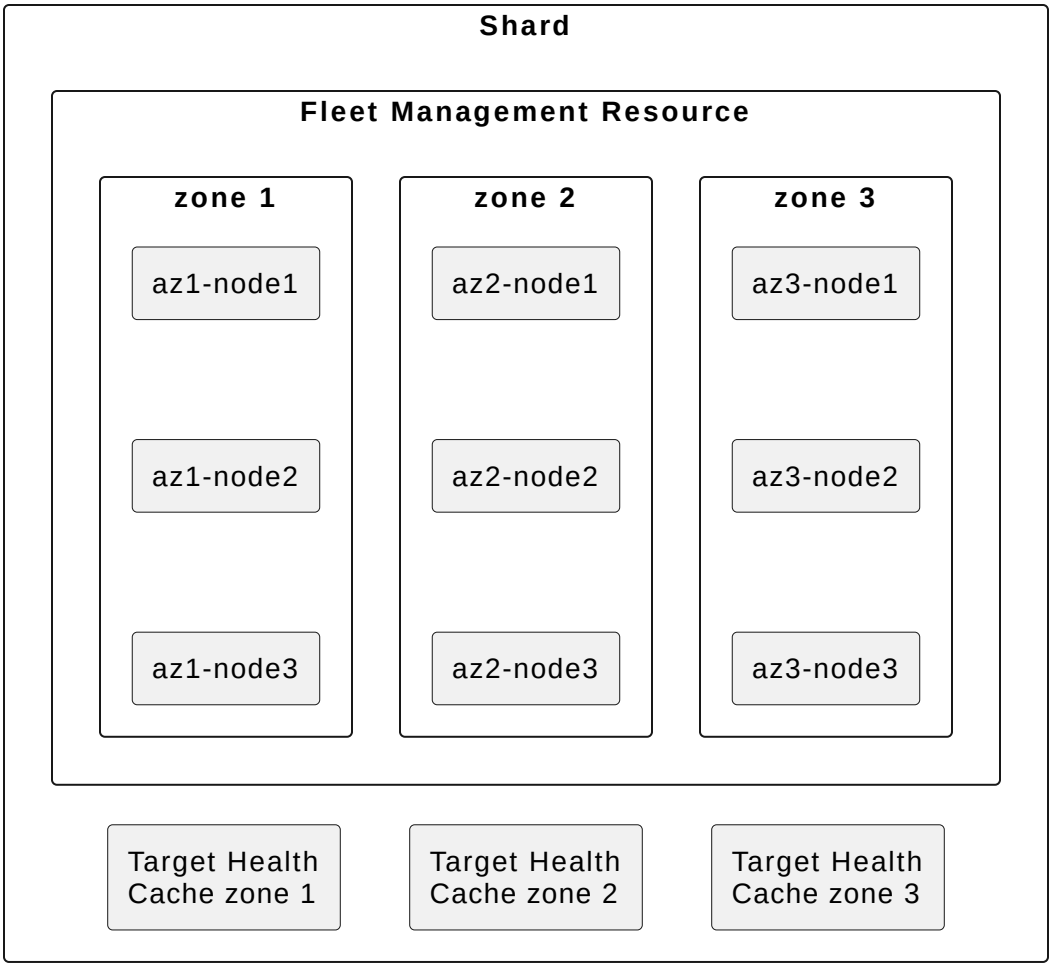


Figure 16. Three-zone shard design

Component overview

Table 4. Components and responsibilities

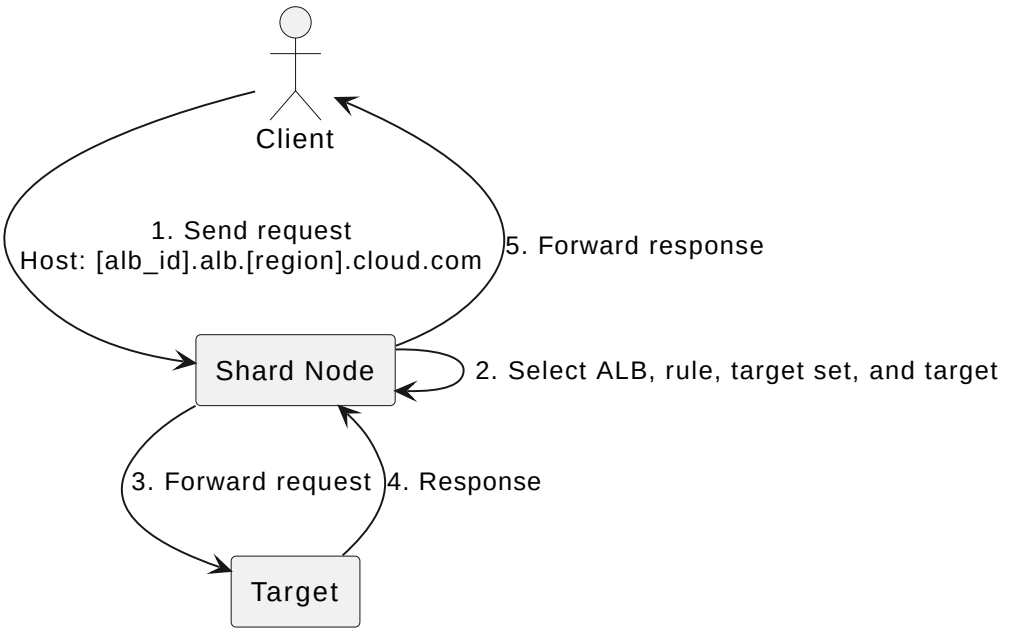
Component	Responsibility
Control plane FE service	Handles client-facing resource creation and updates, validates requests, and persists control-plane intent.
Resource database	Contains the `ApplicationLoadBalancer`, `Rule`, `TargetSet`, `IPTarget`, and `Shard` entities.
Data plane shard	Unit of placement for a tenant's application load balancer.
Configuration service	Generates and distributes configurations.

Component	Responsibility
Shard operations database	Contains the `NodeResource` and `ShardDnsPublication` entities.
Shard nodes	Unit of routing. Serves data-plane traffic by terminating TLS, selecting targets, maintaining backend connections, and forwarding requests.
Fleet management service	Maintains per-shard node counts, autoscales shards, and records node events.
Target health checking subsystem	Checks tenant ALB targets, coordinates probe ownership, writes zonal target-health caches, and supplies shard nodes with routing health state.
Node lifecycle scripts	Node configuration and teardown.
Shard DNS publication reconciler	Derives the desired DNS RRSet from authoritative shard membership and publishes the resulting node IPs.
Observation flow	Carries target-health observations to the control plane.
ALB lifecycle workflows	Workflows for creating and deleting the ALB resource.
Node termination reconciler	Interprets fleet events to identify abrupt node termination.

Critical paths

ALB invoke

1. The client sends a request over the existing connection to a shard node.
2. The shard node fetches local resource configuration.
3. The shard node maps the request to a rule and target set.
4. The shard node selects a target.
5. The shard node forwards the request over an existing connection.
6. The shard node forwards the response.



Create ALB

ALB creation has two critical paths for the synchronous and asynchronous portions. The synchronous path is executed by the control plane FE service. The asynchronous path is executed by the ALB creation workflow.

1. The tenant sends a request to create an ALB resource.
2. Synchronous path: The control plane FE service validates the request, performs authN/authZ and quota checks, creates a resource stub in `CREATING`, triggers the creation workflow, and returns `202 Accepted` to the tenant.
3. Asynchronous path: The creation workflow job samples two random shards, selects the least-loaded shard (using resource count and published node count), persists the shard ID, creates a resource DNS entry, and marks the resource as `ACTIVE`.

Target registration

1. The tenant registers a new target with a target set.
2. The control plane FE service validates the update.
3. The FE service persists the new target in the resource database.

4. The FE service returns `HTTP 200 OK` to the tenant.
5. The configuration service compiles and stores the new target set config delta.
6. It appends a coherent generation to the shard configuration change log.
7. Shard nodes are notified of the new generation.
8. Each node pulls the new config versions.
9. Each node validates and atomically activates the update locally.
10. The node with the partition lease performs health checks on the target.
11. The partition lease holder updates the subshard target-health cache.
12. Subshard nodes pull the target's status from the target-health cache and update their local caches.

Node activation path

1. The startup script is triggered.
2. The startup script loads tenant configurations.
3. The startup script allocates a public IP from the IPAM service.
4. The startup script stands up the execution environment.
5. The startup script creates a `NodeResource` entry and sets the `public_ip`, `public_ip_allocation_id`, and `ready` fields.
6. The shard DNS publication reconciler scans `NodeResource` entries for the shard. It then updates `ShardDnsPublication.desired_members` and `ShardDnsPublication.desired_generation`.
7. The shard DNS publication reconciler updates DNS and waits for the operation to fully complete.
8. The shard DNS publication reconciler updates `ShardDnsPublication.observed_members` and `ShardDnsPublication.observed_generation`.
9. The startup script observes that `ShardDnsPublication` includes the node.
10. The startup script exits.

Shard repair: abrupt loss

In the event of abrupt node loss, we want to quickly update the shard serving set by removing the failed node's IP address. We can ignore member replacement as it's identical to the scale-up critical path.

1. The node termination reconciler sees a node termination event.
2. The node termination reconciler validates with the virtual machine service that the node is terminated and that the node is present in `ShardDnsPublication.observed_members`.
3. The node termination reconciler updates `NodeResource.retirement_requested`.
4. The shard DNS publication reconciler scans `NodeResource` entries for the shard. It then updates `ShardDnsPublication.desired_members` and `ShardDnsPublication.desired_generation`.
5. The shard DNS publication reconciler updates DNS and waits for the operation to fully complete.
6. The shard DNS publication reconciler updates `ShardDnsPublication.observed_members` and `ShardDnsPublication.observed_generation`.
7. The node termination reconciler sees that the node has been removed from `ShardDnsPublication.observed_members`.
8. The node termination reconciler waits for the DNS TTL horizon to elapse so that client caches expire.
9. The node termination reconciler deallocates the public IP address.

Additional critical paths

Critical path	Brief explanation
ALB deletion request	Checks that the ALB is `ACTIVE` and sets the state to `DELETING` as a durable workflow trigger.
ALB deletion workflow	Removes its DNS publication, waits for prior DNS answers and connections to expire, deletes any dangling child resources, and sets the ALB state to `DELETED`, which triggers a configuration tombstone update.

Critical path	Brief explanation
Rule update	The resource database is updated and configuration is distributed.
Node graceful retirement	The termination script signals retirement intent on `NodeResource` and waits for the shard DNS publication reconciler to remove the IP from the shard serving set. After the DNS TTL horizon has passed, the script drains existing connections, releases the IP address, and tears down the node.
Scale-in	The fleet management service identifies excess capacity, selects a node for removal, initiates its graceful retirement path, and terminates the node.
Scale-out	The fleet management service identifies insufficient capacity, provisions a node, and initiates the node activation path.

2.4. Design boundaries

This design omits virtual network integration, keeps virtual machine targets out of scope, and leaves custom certificates out of scope.

This challenge covers routing traffic to targets with public IPs. Routing privately to targets within the cloud requires virtual network integration. This requires coupling with our cloud virtual network implementation and adds significant architectural complexity, and so virtual network integration is deferred.

An obvious target type is a fleet of virtual machines whose target membership tracks fleet membership. Because supporting these targets requires virtual network integration, they are also out of scope.

While TLS termination is supported, tenants can't bring their own certificates. The custom certificate feature is kept out of scope to keep the chapter lean. Custom certificates and their responsibilities are deferred to the API gateway design.